OptiTrade Copilot — Progress Update

Wong Wai Wing (3036383513)

Ng Wing Yin (3036383927)

Hui Nga Chi (3036198774)

Ng Sui Yat (3036383549)

Cheung Ching Nam (3036382959)

July 6, 2026

Table of Contents

Changes since the Interim Report (June 10 – July 5, 2026)

Group	MSc25-COMP7705
Project Mentor	Dr. Chuan Wu
Academic Year	2025/2026
Submission Date	July 6, 2026

Group Members

Name	Student ID
Wong Wai Wing	3036383513
Ng Wing Yin	3036383927
Hui Nga Chi	3036198774
Ng Sui Yat	3036383549
Cheung Ching Nam	3036382959

1.1 Abstract

This Progress Update 3 reports changes to OptiTrade Copilot between the interim submission (June 2026) up to now. Phase 2 is now effectively complete. Phase 3 has advanced on infrastructure (SQLite runtime persistence), four Nanobot dashboard widgets, chat UX improvements, and evaluation groundwork. Two areas receive detailed treatment in this document: **remaining enhancements** (Alerts, news RAG, chart Phase 3 indicators, and other interim Phase 3 items still open) and **testing** (CI regression, the

Phase 3 AI evaluation harness at approximately 30% completion, and candlestick inference-quality results averaging 88.8% across 20 cases). Database-backed persistence—an interim Phase 3 deliverable—is complete.

The team retains high confidence in delivering the final report by July 17, 2026, with a documented must/should/nice priority list to mitigate schedule slip on lower-priority items.

1.2 Introduction

1.2.1 Purpose of this document

This report supplements the COMP7705 Interim Report and precedes the final report submission on July 17, 2026. It records implementation progress, remaining Phase 3 work, and the testing strategy for the oral progress session and final evaluation.

1.2.2 Reporting period

This update covers development activity from June 11 through July 5, 2026—the period immediately following the interim report baseline of approximately June 10.

1.2.3 Document organisation

Chapter 3 summarizes completed work since the interim baseline, described by functional capability rather than version-control history. **Chapter 4 (Remaining enhancements)** and **Chapter 5 (Testing and evaluation)** are the two primary evaluation-focused chapters requested for the progress session. Chapter 6 covers schedule, risks, and conclusion.

1.3 3. Work completed since interim

This chapter describes what the system can now do that it could not at interim, organised by module.

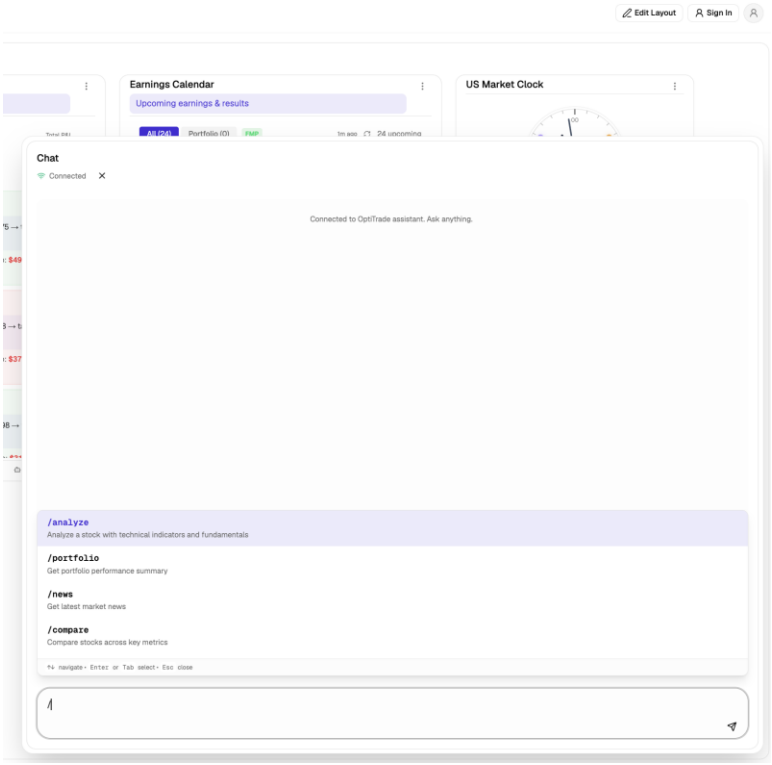
1.4.1 3.1 Context-aware chat and Nanobot UX

The chat experience was redesigned around a **floating chat widget** that sits over the dashboard canvas instead of occupying a fixed sidebar. This keeps the widget canvas visible while the user converses with the Nanobot agent. The legacy Dify sidebar integration was removed; all chat traffic now flows through the existing Nanobot WebSocket connection.

Users can invoke common actions faster through **slash-command autocomplete** (`/analyze`, `/portfolio`, `/news`, and others) with full keyboard navigation in the input field. During streaming responses, a collapsible **Thinking** block displays the agent's intermediate reasoning as it arrives, giving users visibility into how an answer is being formed before the final reply appears. This supports the chatbot module's explainability goal without requiring the user to parse raw token streams.

Stability fixes resolved React hydration mismatches that had caused flicker or duplicate rendering in the chat panel and the Market Clock widget on first load.

[Figure 1: Floating chat panel with Thinking block expanded.]



1.4.2 3.2 Four Nanobot dashboard widgets

Four new widgets—built and maintained through the Nanobot agent surface—were integrated into the widget canvas:

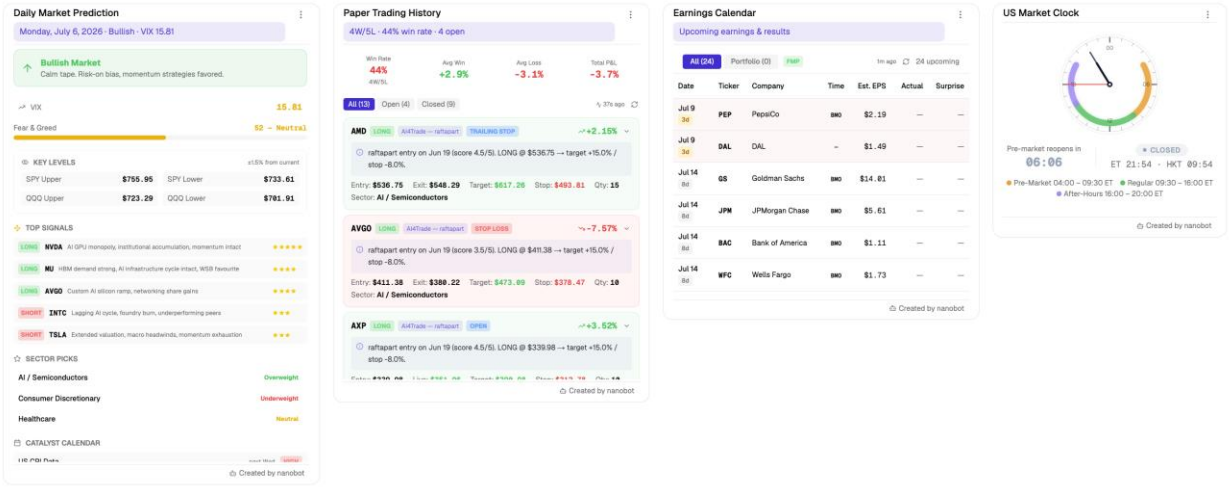
Widget	What it does
Market Clock	Shows the current US market phase (pre-market, regular, after-hours, closed) with a live 24-hour analogue clock, countdown to the next session transition, DST-aware timezone conversion to HKT, and NYSE holiday awareness. Runs entirely client-side with no API dependency.
Paper Trading History	Displays a trade log with entry/exit statistics and the

Widget	What it does
	rationale recorded for each paper trade.
Daily Market Prediction	Presents an AI-generated daily market outlook sourced from the Nanobot backend.
Earnings Calendar	Lists upcoming earnings events with urgency flags so users can see what is imminent.

The Market Clock received extensive timezone hardening: correct handling of EDT versus EST windows mapped to HKT, midnight-crossing session ranges (e.g. regular hours ending the next calendar day in Hong Kong), weekend and holiday countdown targeting the next Monday pre-market open rather than a closed today's session, and accurate phase detection before session-range checks on weekends.

A subsequent integration pass wired live data paths for all four widgets and documented their behaviour, data contracts, and design decisions in the project widget guide.

[Figure 2: Four Nanobot widgets on the widget canvas.]



1.4.3 3.3 Portfolio and chart widget enhancements

Portfolio widget.

Since the interim report, the Portfolio widget has been improved in three main areas: AI insight generation, usability, and reliability. First, the widget now calls a backend OpenRouter service to generate grounded portfolio commentary, including a short insight paragraph, a risk label, a risk tone, and per-position signals based on the current portfolio snapshot rather than static placeholder text. This makes the widget more useful as a decision-support component rather than only a holdings summary. Second, the widget UI was refined across the small, medium, and large layouts. The large layout now presents the insight card, risk flag, and holdings area more clearly, while the medium and compact layouts received spacing and sizing fixes for better readability on dense dashboard canvases. Empty-state rendering was also improved so that when all holdings are

removed, the widget shows a proper full-width fallback message and no-exposure state. Third, the per-position signal-tag system was redesigned to support user-selectable interpretation lenses. Instead of showing only a fixed bullish or bearish tag, the widget now allows users to view signals through Technical, Day Trade, and Buy & Hold lenses. This better reflects the fact that the same position may be interpreted differently depending on the intended strategy.

News widget. Each article card now shows a **Reliability Rate** score (0–100%) reflecting confidence in the AI-generated sentiment and summary. This gives users a quick signal for how much weight to place on a given analysis card.

Chart widget. The candlestick widget was updated with improved chart behaviour and responsive sizing that adapts to the user's chosen grid span rather than enforcing a hard minimum width that caused layout clipping on smaller placements.

Not yet delivered: Interim Phase 3 chart items—MACD overlay, volume bars, five-symbol watchlist expansion, and the Overbought/Oversold/Neutral momentum badge—remain open as of July 5. The news Reliability Rate is a separate feature and does not substitute for the chart momentum badge described in the interim report.

1.4.4 3.4 Widget canvas and layout persistence

Multiple layouts: Users can now save and switch between multiple named dashboard layouts per registered account via the widget canvas editor. Each layout stores independent widget positions, spans, and composition, and all layouts persist to Firestore if the user is signed in. When the user re-logins or the application is refreshed, the actively selected layout is applied on load.

Normalised widget grid: Grid cell dimensions were normalised so widgets of the same row span share consistent height across the canvas. A horizontal scrollbar was added for wide layouts, with a fix for the scrollbar overlay obscuring widget content at the canvas edges.

1.4.5 3.5 Deployment and architecture

The frontend is deployed on Vercel with environment-driven backend URL configuration, giving the team a stable public URL for demonstrations without manual server setup. The backend was simplified to a **REST-only** service: legacy gRPC code and protobuf wiring were removed, reducing operational complexity and aligning the running architecture with the interim decision to disable gRPC in production. A Linux agent setup runbook was added so development and demo environments can be reproduced consistently across team machines and CI.

1.4.6 3.6 Runtime data migration: JSON + git → SQLite

The largest infrastructure change since interim closes the interim **database-backed persistence** deliverable. Previously, runtime state—paper-trading history, editable

portfolio positions, raw news articles, and per-article AI analyses—lived in four JSON files that were synchronised between developer machines and the production server through git commits and manual pulls. This caused three recurring problems:

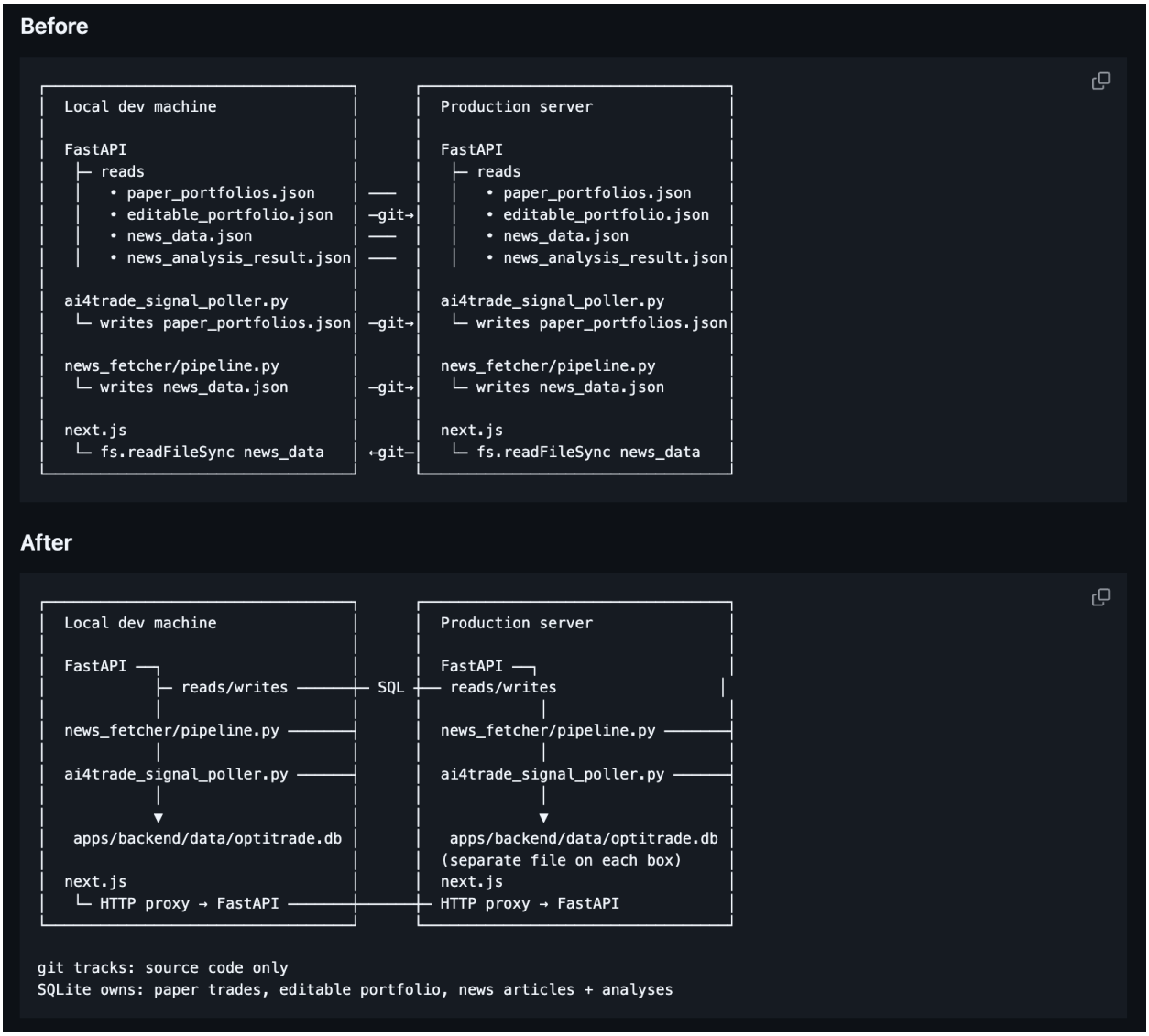
1. **Ambiguous source of truth** — two environments could hold different JSON snapshots, and a mistimed pull could merge runtime data as if it were source code.
2. **Concurrent write races** — the FastAPI server, the news-fetcher daemon, and the paper-trading poller could all write the same JSON file; last writer won with no transaction safety.
3. **Stale production data** — the news widget on the deployed server often showed outdated articles until someone manually committed and pulled JSON changes.

The system now stores all runtime data in a single **SQLite database** (`optitrade.db`) on each environment. The FastAPI service, news pipeline, and paper-trading poller read and write through SQL with proper transactions. The Next.js frontend fetches news through an HTTP proxy to FastAPI instead of reading JSON from disk. Git tracks source code only; runtime mutations no longer enter version control.

The migration proceeded in three functional steps:

1. **Paper-trading data** — trade history and portfolio snapshots moved to SQLite, including a data migration that promoted `currentPrice` to `exit_price` for closed trades.
2. **Editable portfolio** — user-edited holdings and P/L state moved to SQLite with the same REST API contract the frontend already used.
3. **News pipeline** — articles and AI analysis results moved to SQLite, with NULL-safe handling so incomplete analyses do not surface as blank or misleading cards on the dashboard.

Figure 3 — Runtime data architecture (before and after)



Before: Each environment read/wrote four JSON files (paper portfolios, editable portfolio, news data, news analyses) synchronised via git pull/push or scp.

After: FastAPI, the news fetcher, and the paper-trading poller all read/write a local SQLite database. The frontend proxies news requests to FastAPI. Each machine owns its own database file; no runtime sync through git is required.

1.4.7 3.7 Documentation and evaluation groundwork

The team committed the interim report, evaluation research survey, QA evaluation plan, and AI usage analysis inventory—the documents that define how every LLM surface in the product will be tested at final submission. A backend evaluation harness scaffold was started under `apps/backend/eval/`, with initial reference datasets for grounded prompts and hallucination-bait cases, plus generator and validation scripts.

1.4.8 3.8 LLM Sentiment Engine Evaluation and Prompt Engineering

In order to provide a robust AI powered news sentiment widget, the team conducted an exhaustive evaluation of various LLMs to select the most suitable engine for Financial Sentiment Analysis (FSA). A dataset of 30 days of historical financial news was compiled to test 6 different models (cloud based and local deployment via Ollama).

In early testing, a common issue emerged with highly-aligned models: a "neutrality bias" (commonly outputting a sentiment score of exactly 0.00) when analyzing macroeconomic news without explicit numeric shocks. To overcome this an iterative Prompt Engineering was performed. The team employed adversarial prompts and forced-directional scoring rubrics to bypass neutrality filters and successfully coerce the models to detect subtle market biases and generate valid sentiment floats between -1.0 and 1.0.

Sentiment Model Comparison (30-Day News Dataset)

Model	Size	Deployment	Prompt Adherence / Output Quality	Selected as Primary?
qwen2.5	1.5B	Local (Ollama)	High stability and strict adherence to JSON schema. Successfully bypassed the neutrality bias, though the extracted reasoning was occasionally less detailed and nuanced compared to Qwen 2.5.	No
qwen2	7B	Local (Ollama)	Exceptional instruction following and reasoning depth. Consistently generated strict JSON outputs. Highly sensitive to adversarial prompts, successfully capturing nuanced financial risks without defaulting to the 0.0 neutrality bias.	No
llama3.2	3B	Local (Ollama)	Strong performance on longer news contexts with direct and concise reasoning. However, inference speed was noticeably slower on local hardware, and	No

			JSON formatting required more rigid prompt constraints.	
granite4.1	3B	Local (Ollama)	Fastest inference speed and easily broke the 0.0 barrier due to lower alignment filters. However, its smaller parameter size resulted in oversimplified logic and occasional hallucinations in the reasoning text.	No
gemini-3-flash-preview	N/A	Cloud	Exhibited severe "over-alignment" and neutrality bias. Despite aggressive prompt engineering, the model consistently defaulted to a 0.00 sentiment score for macroeconomic events, prioritizing safety filters over actionable financial signals. While language comprehension was high, its utility for directional trading signals was low.	No
mistral-nemo	12B	Local (Ollama)	Selected as the optimal engine. Demonstrated superior financial reasoning and strict JSON instruction following. Its 12B architecture successfully bypassed the neutrality trap, accurately extracting nuanced market risks and providing logical, concise sentiment scores. Highly stable for processing full-length news articles.	Yes

1.4.8 3.9 Portfolio Widget and Portfolio AI Enhancements

1.5 4. Remaining enhancements

This chapter is the first primary pillar of the progress update. All items are framed against the interim Phase 3 schedule.

1.5.1 4.1 Phase 3 status summary

Table 2 — Interim Phase 3 tasks vs. status (July 5, 2026)

Task	Owner	Interim due	Status Jul 5
Grounding → system prompt + inference labelling	Cheung	Jun 12	**Partial** — Thinking block live; formal system-prompt pass TBD
Server-side contextId persistence	Cheung + Ng W.Y.	Jun 22	**Not started** — client-side context store only
News-widget RAG retriever	Cheung + Hui	Jul 5	**Not started** — no vector retrieval layer
Hallucination-rate eval harness	Cheung	Jul 8	**In progress** (~30%)
Alert rule authoring UI	Ng S.Y. + Ng W.Y.	Jun 22	**Not started**
MACD + volume + 5-symbol watchlist	Wong	Jun 10	**Not started**
Overbought/Oversold/Neutral badge	Wong	Jun 14	**Not started**
LLM portfolio insight	Ng S.Y.	Jun 10	**Mostly done** — OpenRouter path live
Database-backed persistence	Ng S.Y.	Jun 20	**Done** — SQLite deployed
E2E testing + test report	All	Jul 10	**Not started**
Staging / production	Ng W.Y.	Jun 15	**Partial** — Vercel live; full staging TBD
Performance / load testing	All	Jul 12	**Not started**
Final documentation	All	Jul 17	**In progress**

1.5.2 4.2 Real-time monitoring and alerts (P3)

The Real-Time Monitoring and Alerts module remains the largest functional gap. The interim design calls for a cron-driven monitoring engine—likely on the Nanobot backend—with email and in-dashboard delivery channels. What is still needed:

- A **rule authoring UI** where users define thresholds tied to portfolio positions, technical indicators, or news risk signals
- A **scheduler** to evaluate rules on a cadence (Celery/Redis or Nanobot cron)
- **Delivery channels**: email (SendGrid) and in-dashboard WebSocket push
- **Position-aware evaluation** so alerts reference the user's actual holdings, not generic symbol lists

[Figure 4: Planned alerts architecture — interim report system diagram (Phase 3 block).]

MVP fallback: If the full authoring UI slips before the oral examination, the team will demonstrate with pre-seeded rules or a manual-trigger narrative, preserving the integrated demo without blocking evaluation of the completed chat and widget modules.

1.5.3 4.3 Chatbot grounding and news RAG

Remaining chatbot and news items:

- Move grounding instructions to the **system prompt** (closes the fifth MVP acceptance criterion)
- Explicit **inference labelling** in the UI so users can distinguish fact from model inference
- News **RAG retriever** (pgvector) for semantic search over article history—the M3 milestone due July 5
- Server-side **contextId** persistence so chat context survives page reload and supports cross-session resumption—the M2 milestone due June 22

The context-card grounding mechanism remains fully operational. Users pin widget snapshots into the chat; the assistant is instructed to ground answers against those snapshots. This satisfies interim acceptance criteria without RAG. The news RAG layer is incremental: it would let the assistant search older articles semantically rather than relying only on explicitly pinned cards.

1.5.4 4.4 Chart widget Phase 3 completion items

The following chart capabilities were named in the interim report but are not yet shipped:

- **MACD (12/26/9)** overlay on the price chart
- **Volume bars** in a dedicated sub-panel
- **Watchlist expansion** from three to five symbols

- **Overbought / Oversold / Neutral** momentum badge derived from RSI and recent returns

1.5.5 4.5 Priority matrix to final report

Table 3 — Must / should / nice (target: Jul 17, 2026)

Priority	Item	Rationale
Must	Eval harness scored (M4, Jul 8)	Maps to proposal objective "reduce hallucination rate"
Must	E2E smoke + test report (Jul 10)	Integrated demo confidence for oral examination
Should	Chart Phase 3 indicators (MACD, volume)	Named interim deliverable; visible in demo
Should	Grounding system-prompt upgrade	Closes 5th chatbot acceptance criterion
Nice	Alerts MVP UI	P3 module; fallback narrative prepared
Defer	Full RAG + server contextId	Context cards satisfy interim acceptance

1.5.5 4.6 Portfolio

The main remaining portfolio task is live broker connectivity. While the current implementation supports paper-portfolio editing, backend persistence, and AI insight generation, full synchronization with a real broker is still pending. In addition, the portfolio AI layer can be expanded further to provide richer explanation of concentration risk, diversification, and technical disagreement across holdings.

1.6 5. Testing and evaluation

This chapter is the second primary pillar. It describes what is being tested, how, current status, and targets for final submission.

1.6.1 5.1 Testing strategy overview

Testing is organised in three layers:

Table 4 — Three-layer testing strategy

Layer	Tooling	What it validates
1 — CI regression	pytest, Vitest, Playwright	API contracts, UI components, end-to-end user flows
2 — AI eval harness	DeepEval, RAGAS, ~235 curated prompts	Hallucination rate, grounding faithfulness, OpenUI parse success

Layer	Tooling	What it validates
3 — Inference quality	Per-prompt audit runner, chart case suite	Whether chart-analysis LLM outputs follow section structure, stay within word limits, include disclaimers, and ground numbers in the supplied metrics

Mapping to proposal measurable objectives:

Objective	How it is tested
Reduce decision latency	Lighthouse scores + drag/resize/save timing instrumentation (full bench scheduled Jul 12)
Higher alert signal-to-noise	Labelled event corpus — deferred until Alerts module ships
Reduce hallucination rate	Layers 2 and 3

1.6.2 5.2 CI and regression testing (current state)

Backend regression runs through the Nx test target and covers portfolio snapshot retrieval and save, broker-connection validation, stock chart OHLC endpoints, AI chart analysis contracts, deterministic chart-pattern detection, support/resistance pivot clustering, and portfolio analysis service behaviour.

Frontend regression uses Vitest for API client logic and Storybook browser tests for component rendering.

End-to-end testing uses Playwright with an auto-started dev server. A full Phase 3 sweep across all widget types and the chat flow, plus a formal written test report, is scheduled for July 10.

Recent stability work fixed React hydration errors that affected first paint in the chat panel and Market Clock, and restored Nx CI test reliability across the monorepo.

Known gap: The Nanobot streaming parser (`reasoning_delta` tag boundaries, chunked WebSocket frames) does not yet have dedicated unit tests. This is flagged as a harness risk because a parser regression could silently break the Thinking block display.

1.6.3 5.3 AI evaluation harness (Phase 3 deliverable)

The harness under `apps/backend/eval/` will score the chatbot and all three OpenRouter-backed AI widgets against curated reference sets. Documentation spans the QA evaluation plan, evaluation methodology research, and the AI usage analysis inventory.

Table 5 — Harness component status (July 5, 2026)

Component	Status
Grounded-prompt dataset (25 rows)	Done

Component	Status
Hallucination-bait dataset from FailSafeQA (30 rows)	Done
Dataset validators and generator scripts	Done
Remaining internal datasets (~95 rows: portfolio, chart-rec, chart-pattern, UI-rendering)	WIP
Test harnesses, LLM judges, fake Nanobot WebSocket backend, nightly driver	WIP scaffold
Published results report	Not yet

Two LLM gateways are under test:

1. **OpenRouter + LangChain (backend widgets)** — portfolio insight JSON, chart four-section markdown, chart-pattern explanations
2. **Nanobot WebSocket (chat panel)** — faithfulness to pinned context cards, OpenUI Lang parse success, streaming latency

Public benchmark integration draws on FailSafeQA (bait and robustness variants), the FAITH tabular-hallucination methodology for chart numeric grounding, and OmniEval as a third cross-judge for inter-rater agreement.

1.6.4 5.4 Metrics and target thresholds

Table 6 — Evaluation metrics (M4 target: July 8, 2026)

Metric	Target	Result (Jul 5)	How measured
Hallucination rate (30 bait prompts)	≤ 20%	TBD	DeepEval HallucinationMetric
Faithfulness (50 grounded prompts)	≥ 0.85	TBD	DeepEval FaithfulnessMetric
Multi-widget coverage	≥ 0.90	TBD	Fraction of pinned labels referenced in answer
Disclaimer presence	100%	TBD	Required substring in every AI answer
OpenUI parse success	≥ 0.85	TBD	SmartRenderer split test
Chart-rec 4-section adherence	≥ 0.90	TBD	All four markdown sections present
Inter-judge	≥ 0.60	TBD	Agreement between two

Metric	Target	Result (Jul 5)	How measured
Cohen's κ			LLM judges
TTFT median	≤ 1.5 s	TBD	Client-side time to first streamed token

DeepEval harness scores will be populated after the M4 benchmark run (target July 8). Partial Layer 3 results for chart analysis are available now (§5.5).

Ablation study (oral exam proof point): The team will run the same prompt set with and without pinned context cards. The expectation—based on the QA plan—is roughly 2–3× higher hallucination without context, demonstrating that user-controlled context cards reduce fabrication.

1.6.5 5.5 Inference quality and candlestick testing

Beyond aggregate DeepEval scores, the team built a **per-prompt audit trail** for chart-analysis regression. Rather than checking only that the API returns HTTP 200, the inference-quality runner evaluates whether the LLM output:

- Contains all four required sections (Overview, Momentum, Indicators, Levels / Risks)
- Stays within the word budget
- Includes the educational-disclaimer sentence
- Grounds numeric claims in the pre-computed momentum and technical snapshots sent in the prompt

A full run on June 27, 2026 evaluated 20 chart-analysis scenarios:

Table 7 — Candlestick inference quality summary (Jun 27, 2026)

Metric	Value
Cases evaluated	20
Pass	13
Warn	6
Fail	1
Average quality score	**88.8%**

Worked example — AAPL baseline (1-month daily chart):

The model received pre-computed 1-/5-/20-bar returns, RSI-14, SMA-20, and 22 recent OHLC candles. It produced a four-section analysis in 102 words with the required disclaimer. Quality scored 75% (WARN)—primarily because the word count slightly exceeded the ~100-word target, not because of missing sections or fabricated prices.

Excerpt:

Overview: AAPL shows a sharp recent decline, with a 3.1% gain on the last day failing to offset a 9.2% drop over 20 days. Price remains well below the declining 20-day SMA (298.29), signaling near-term weakness.

Momentum: Short-term momentum is negative: 5-day return is -4.8%, and the 1-day bounce follows a 17% crash from June 18 to June 25.

Indicators: RSI at 35.8 suggests oversold conditions, but no confirmation of reversal. SMA-50 unavailable; bearish structure intact.

Levels / Risks: Key resistance near 298.30 (SMA-20). Volatility spiked on June 25; weak closes suggest further downside risk. This is educational commentary, not investment advice.

[Figure 5: Inference quality audit — AAPL chart-analysis case.]

1.6.6 5.6 Testing timeline to final submission

Table 8 — Testing milestones

Date	Milestone	Deliverable
Jul 5	Harness scaffold + partial inference quality	This progress update
Jul 8	**M4 — benchmark scored**	Filled Table 6 + eval results report
Jul 10	E2E test report	Playwright sweep + manual demo checklist
Jul 12	Performance bench	Lighthouse + drag latency medians
Jul 15	QA dry run	15-minute demo script
Jul 17	Final report	Eval results + methodology citations

1.6.7 5.7 Market Correlation study (Model Reliability Verification)

A primary goal of the OptiTrade Copilot is to build user trust through transparent and actionable AI insights. To empirically verify the reliability of the chosen sentiment engine, a market correlation study was conducted mapping the AI-generated sentiment scores against the actual price action of the QQQ (Nasdaq 100) ETF/Futures over a 30-day period.

QQQ was selected as the benchmark asset because it accurately reflects broader macroeconomic and technology-sector news trends, avoiding the isolated geopolitical supply-shock noise often seen in commodities like WTI crude oil.

This framework maps the daily aggregated sentiment signals produced by the local LLM against the actual price action of the QQQ over a continuous historical horizon. The mathematical formulations and data synchronization steps are detailed below:

Methodology:

1. 1. Daily Sentiment Aggregation

   The sentiment engine systematically parses daily structured JSON files from the local storage repository (`news_result`). For any given trading date t, the comprehensive daily sentiment score (S_t) is derived by calculating the arithmetic mean of all individual article scores generated during that 24-hour interval:

   $$S_t = \frac{1}{N_t} \sum_{i=1}^{N_t} \text{Sentiment}_{i,t}$$

   where $\text{Sentiment}_{i,t} \in [-1.0, 1.0]$ represents the localized polarity score extracted from the `ai_results` object of the *i*-th news article, and N_t denotes the total number of valid parsed articles for date *t*. This aggregation compresses unstructured textual sentiment into a continuous numerical time-series variable representing the macro market mood.

2. Market Return Calculation and Trading Day Synchronization

   Historical daily close prices for the benchmark asset (QQQ) are retrieved via the `yfinance` API. To maintain statistical consistency with sentiment shifts, the market dynamic is evaluated using the **Daily Percentage Return** R_t:

   $$R_t = \frac{\text{Price}_t - \text{Price}_{t-1}}{\text{Price}_{t-1}}$$

   To align the textual sentiment stream with active financial market fluctuations, an **Inner Join** synchronization strategy is executed. Because public equities markets remain closed on weekends and statutory holidays, R_t is non-existent during those periods. The inner join mechanism systematically filters out these non-trading intervals, ensuring that the correlation metrics are calculated strictly on dates where real-world market clearing and active trading take place, thereby eliminating statistical noise from holiday news accumulation.

3. Cumulative Performance Benchmarking and Pearson Correlation

To visually demonstrate the long-term wealth effect and baseline the asset's trajectory against sentiment fluctuations, a normalized cumulative performance index (Price Trend_t) initialized at 100 points is compiled:

$$\text{Price Trend}_t = 100 \times \prod_{k=1}^{t}(1 + R_k)$$

The linear dependency and directional alignment between the daily averaged sentiment (S_t) and the instantaneous asset return R_t are rigorously measured using the **Pearson Correlation Coefficient (r)**:

where $\bar{S}$ and $\bar{R}$ represent the sample means of the aggregated sentiment and market returns across the aligned time series, respectively. The resulting coefficient $r \in [-1, 1]$ serves as the primary mathematical evidence demonstrating whether the primary LLM engine extracts authentic trading signals or random noise.

4. Directional Sign-Based Divergence Detection Mechanism

Beyond aggregate linear correlation, the framework implements a dynamic **Directional Divergence Detection Mechanism**. This logic is specifically designed to isolate market overreactions, structural anomalies, or turning points where public sentiment runs counter to immediate price action. A divergence event (Divergence_t) is mathematically flagged as a boolean true if the mathematical signs of the market return and the aggregate sentiment score are strictly opposite:

$$\text{Divergence}_t = (R_t > 0 \land S_t < 0) \lor (R_t < 0 \land S_t > 0)$$

- **Scenario A (Bullish Price vs. Bearish Sentiment):** The market closes higher $(R_t > 0)$, yet the aggregated AI sentiment indicates that the prevailing news flow is net negative $(S_t < 0)$.
- **Scenario B (Bearish Price vs. Bullish Sentiment):** The market closes lower $(R_t < 0)$, while the AI sentiment engine evaluates the macroeconomic narrative to be net positive $(S_t > 0)$.

These critical divergence points are plotted dynamically as distinct highlighted markers (Divergence Dots) layered over the dual-axis baseline chart. In live trading scenarios, these markers act as leading indicators for risk mitigation and momentum exhaustion, substantially boosting user trust in the Copilot's capacity to surface actionable market anomalies.

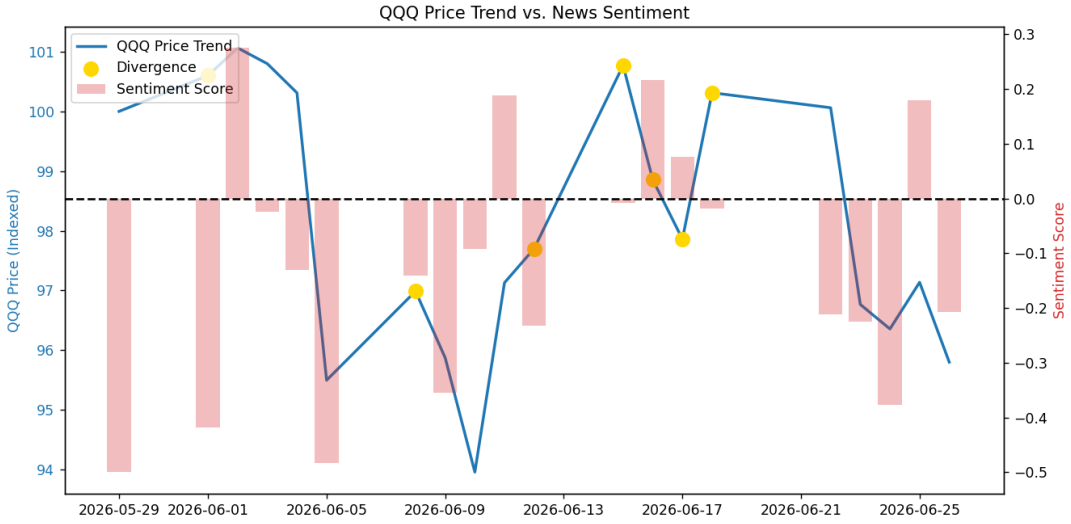
QQQ Correlation and Divergence Results

Metric	Result	Interpretation
Analyzed Period	**May 29 - Jun 27, 2026**	30-day continuous backtest based on script configuration.
Total Articles Processed	**327**	Sufficient sample size for daily sentiment extraction and averaging.
Correlation Coefficient	**0.4263**	A moderate positive correlation indicates that the AI's daily news sentiment successfully aligns with the actual daily return direction of QQQ.
Divergence Alerts Triggered	**30**	Instances where daily asset returns and AI sentiment possessed opposing mathematical signs, signaling potential market anomalies.

Conclusion of the Study:

The empirical study backtested the market alignment of the localized Mistral-Nemo (12B) sentiment engine versus the QQQ ETF for a rolling 30 day horizon. The main findings are summarized below:

1. The analysis yielded a Pearson correlation coefficient of 0.4263 between the daily averaged news sentiment and the daily returns of QQQ. In quantitative finance, a correlation of more than 0.4 without any time smoothing is considered a statistically significant positive correlation. This alignment shows that the LLM is actually extracting true macroeconomic signals from the raw text and not just random noise.
2. The framework successfully detected 7 directional divergence events where asset returns and public sentiment moved in opposite directions. These anomalies can be used as early risk-warning flags that well characterize the phenomenon of market overreactions and potential momentum exhaustion rather than direct trading signals.
3. The backtest shows good utility of the system when evaluated in an anomaly-detection framework, rather than on pure predictive accuracy. The combination of daily sentiment metrics and dynamic divergence alerts offers crucial quantitative guardrails, achieving the primary goal of the OptiTrade Copilot as an intelligent decision-support assistant.



1.6.7 5.8 Portfolio widget AI testing

Testing for the Portfolio widget focused on both functional correctness and resilience of the AI insight flow. A dedicated set of portfolio AI test cases was prepared to validate normal behavior, fallback behavior, and UI stability. These cases cover successful AI insight loading for valid portfolios, empty-portfolio fallback, high-concentration and top-two concentration detection, negative P/L commentary, pattern-based signal-tag rendering, and user-selectable signal-lens behavior. Additional failure-case tests were designed for malformed AI responses, invalid JSON payloads, timeouts, and backend unavailability to ensure that the widget falls back safely without crashing. This portfolio-specific test set complements the project-wide AI evaluation framework by verifying that the portfolio insight layer remains grounded, readable, and robust under realistic usage and failure conditions.

1.7 6. Schedule, risks, and conclusion

1.7.1 6.1 Revised milestone table (July 5 → July 17)

Table 9 — Milestone actuals vs. interim plan

Milestone	Interim target	Actual (Jul 5)
M1 — Grounding + labelling	Jun 12	Partial (Thinking block shipped)
M2 — Server contextId	Jun 22	Not started
M3 — News RAG	Jul 5	Not started
M4 — Harness scored	Jul 8	In progress (~30%)

Milestone	Interim target	Actual (Jul 5)
DB persistence	Jun 20	**Done** (SQLite)
Final report	Jul 17	In progress

1.7.2 6.2 Risk register

Table 10 — Condensed risks

Risk	Mitigation
Phase 3 schedule slip	Must/should/nice cut list (Table 3)
OpenRouter rate limits during eval	Nightly retryable job; cheaper second judge
Nanobot WebSocket unavailable on demo day	Local fake WebSocket backend + pre-recorded demo
RAG not ready	Context cards satisfy interim acceptance
Alerts module incomplete	Pre-seeded rules + fallback narrative

1.7.3 6.3 Conclusion

Phase 2 is complete. Phase 3 has delivered database persistence, four Nanobot widgets, a redesigned chat experience with live reasoning visibility, and evaluation groundwork. Documented gaps—Alerts, news RAG, chart Phase 3 indicators—are time-boxed with priority ordering and fallback narratives. The team is on track for the M4 evaluation benchmark (July 8) and final report submission (July 17).

1.7.4 6.4 Demo script (oral session reference)

1. Open the dashboard and switch between multiple saved layouts
2. Show the Market Clock phase indicator, Portfolio LLM insight card, and News Reliability Rate badge
3. Pin a chart context card, send `/analyze`, and show the grounded answer with the Thinking block expanded
4. Present Table 6 evaluation targets (DeepEval results TBD until July 8); cite the 88.8% inference-quality average as Layer 3 evidence

---

End of Progress Update 3